Refactorización del script en funciones reutilizables (Bloque 6) / Refactoriza tu script en un módulo del repositorio

✓ **Hecho:** Ver

UNIDAD U1.3 · Python aplicado para integración con IA

Refactoriza tu script en un módulo del repositorio

Ya tienes claro qué significa **refactorizar**, qué responsabilidades separar, y qué tiene que seguir exactamente igual cuando termines. Esta lección es donde aplicas todo eso, directamente sobre tu propio módulo, dentro de tu repositorio.

> No estás partiendo de cero: estás tomando la función que ya construiste en el **Bloque 5**, y reorganizándola. Al final de esta lección, tu **módulo** va a hacer exactamente lo mismo que hoy, pero repartido en funciones más pequeñas y con nombres claros.

El orden en que hagas este trabajo también importa, no solo el resultado final. Ir **función por función**, verificando en cada paso, es lo que separa una refactorización tranquila de una donde terminas con tres cosas rotas a la vez y sin saber cuál causó qué.

Antes de empezar, confirma que tu entorno virtual está activo (el mismo que usaste en el Bloque 5): no necesitas instalar nada nuevo, porque vas a seguir usando las mismas dependencias ("requests" y "python-dotenv") y el mismo archivo ".env" que ya tenías configurados.

Antes de tocar nada: asegura un punto de partida

Antes de mover una sola línea de código, confirma que tu versión actual (la del Bloque 5) está guardada en un **commit**. Si tienes cambios sin confirmar, ciérralos con un **commit** primero.

> ↩️ **Esto no es un trámite**: te da un punto exacto al que volver si algo sale mal a mitad de la **refactorización**. Refactorizar sin un punto de partida seguro es mucho más arriesgado de lo que parece.

Piensa en lo que pasaría sin ese resguardo: si a mitad de mover código algo deja de funcionar y ya no recuerdas exactamente qué tenías antes, terminas reconstruyendo de memoria en vez de simplemente volver atrás. Un **commit** reciente evita ese escenario por completo: si algo sale mal, siempre puedes descartar los cambios y retomar desde ahí.

🔄 Refactoriza una función a la vez, no las tres juntas

Aunque el objetivo final son tres funciones nuevas, no las escribas ni las conectes todas de un solo intento.

> Extrae la primera, verifica que tu módulo sigue funcionando igual, y recién entonces avanza a la siguiente.

Esta forma de avanzar, de a un cambio pequeño y verificado a la vez, es una práctica que te va a servir mucho más allá de esta lección: cuanto más pequeño es el paso que das antes de comprobar que todo sigue bien, más fácil es encontrar el origen de un problema si algo se rompe.

🧩 Divide tu función en las tres responsabilidades



Crea una función para armar la petición

Toma esa parte de tu código actual y muévela a una función nueva (por ejemplo, `armar_peticion(pregunta)`), que lea tu credencial, arme la **URL** y el cuerpo de la petición, y devuelva ambos.



Crea una función para enviar la petición y devolver la respuesta ya convertida

Esta función (por ejemplo, `enviar_peticion(url, body)`) solo se encarga de llamar a `requests.post` y devolver el resultado de `.json()`, sin extraer todavía el texto específico.

específico.



Crea una función para extraer el dato que te interesa

Esta función (por ejemplo, `extraer_texto(datos)`) recibe la respuesta ya convertida a diccionario, y navega esa estructura hasta devolver el texto generado.



Ajusta la función principal para que las use en orden

Tu función original ahora se reduce a llamar, en secuencia, a las tres funciones nuevas: `armar_peticion`, luego `enviar_peticion`, luego `extraer_texto`, y devolver ese resultado final.



Borra el código viejo que ya quedó duplicado

Cualquier línea que hayas movido a una función nueva no debería seguir existiendo, además, en el lugar original.

Al terminar los cinco pasos, tu archivo completo debería verse así:

```python
import os
import requests
from dotenv import load_dotenv

 load_dotenv()

 def armar_peticion(pregunta):
     api_key = os.getenv("GEMINI_API_KEY")
     modelo = "gemini-3.6-flash"
     url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateConte
     body = {"contents": [{"parts": [{"text": pregunta}]}]}
     return url, body

 def enviar_peticion(url, body):
     respuesta = requests.post(url, json=body)
     return respuesta.json()

 def extraer_texto(datos):
     return datos["candidates"][0]["content"]["parts"][0]["text"]

 def obtener_dato_de_prueba(pregunta):
     url, body = armar_peticion(pregunta)
     datos = enviar_peticion(url, body)
     return extraer_texto(datos)
```

Si tu proveedor no es **Gemini**, los nombres de función y el orden en que se llaman son igual de válidos: lo único que cambia es qué hace cada una por dentro (la URL, el formato del cuerpo, los nombres de campo de la respuesta).

🔎 Cómo confirmar que el comportamiento no cambió

Antes de refactorizar

Antes de refactorizar, ejecuta tu función con el mismo caso que usaste en el **Bloque 5**, y anota el resultado exacto que devuelve.

Después de refactorizar

Después de refactorizar, ejecútala de nuevo con ese mismo caso. Si el resultado es idéntico al que anotaste, la **refactorización**

salió bien. Si es distinto, hay un **error** introducido en el camino, y conviene revisar función por función hasta encontrarlo antes de seguir.

En términos concretos, esto se ve así. Antes de mover cualquier código, desde la raíz de tu proyecto:

```
python src/consumir_gemini.py
Sí, esta conexión funciona correctamente.
```

El texto exacto de esa respuesta va a variar en cada ejecución, porque proviene de un modelo generativo real; lo que tienes que anotar no es la palabra por palabra, sino que la llamada haya funcionado sin errores. Ejecuta exactamente el mismo comando otra vez apenas termines de refactorizar: si vuelves a obtener una respuesta válida, sin errores nuevos, para la misma pregunta, la refactorización salió bien.

🚨 Qué hacer si el resultado cambió

Si el resultado no coincide, no sigas avanzando ni intentes arreglarlo a ciegas. Vuelve a tu punto de partida seguro (el **commit** del **Bloque 5**) y repite la extracción de a una función, comprobando después de cada una.

Una técnica simple ayuda bastante aquí: agrega un print temporal justo después de cada función nueva, mostrando lo que devuelve. Comparar esos valores intermedios contra lo que esperabas suele señalar exactamente en qué función quedó el problema, sin tener que revisar todo el módulo de una vez.

> ☝️ Recuerda quitar esos print temporales antes de tu commit final.

Modulo Refactorizado

Mismo resultado, código más limpio y organizado.

consumir_gemini.py

src > consumir_gemini.py > obtener_dato_de_prueba

```python
import os
import requests
from dotenv import load_dotenv

load_dotenv()

def armar_peticion(pregunta):
    api_key = os.getenv("GEMINI_API_KEY")
    modelo = "gemini-3.6-flash"
    url = f"https://generativelanguage.googleapis.com/v1beta/models/{modelo}:generateContent?key={api_key}"
    body = {"contents": [{"parts": [{"text": pregunta}]}]}
    return url, body

def enviar_peticion(url, body):
    respuesta = requests.post(url, json=body)
    return respuesta.json()

def extraer_texto(datos):
    return datos["candidates"][0]["content"]["parts"][0]["text"]

def obtener_dato_de_prueba(pregunta):
    url, body = armar_peticion(pregunta)
    datos = enviar_peticion(url, body)
    return extraer_texto(datos)

print(obtener_dato_de_prueba("Confirma en una frase corta que esta conexión funciona"))
```

Problems Output Debug Console Terminal Ports powershell

```
(venv) PS C:\Users\crist\OneDrive\Escritorio\Nueva carpeta\Nuevo Proyecto> python src/consumir_gemini.py
Confirmado, la conexión funciona correctamente.
(venv) PS C:\Users\crist\OneDrive\Escritorio\Nueva carpeta\Nuevo Proyecto>
```

⚠ Errores comunes al hacer este tipo de cambio

Un dato que la función nueva necesita, pero no recibe.

Al mover código a una función separada, algo que antes estaba disponible en el mismo bloque (una variable, un valor) puede dejar de estar visible. Hay que pasarlo explícitamente como parámetro.

Cambiar sin querer el orden de las operaciones.

Si el orden en que se llaman las funciones nuevas no es el mismo que tenía el código original, el resultado puede cambiar aunque cada función individual esté bien escrita.

Dejar código duplicado sin darte cuenta.

Un fragmento que quedó copiado en dos lugares a la vez es una fuente casi garantizada de errores más adelante, cuando alguien corrija uno de los dos y olvide el otro.

Nombres de función que no coinciden con lo que hacen.

Si renombras una función a mitad del proceso, revisa que todos los lugares donde se llama queden actualizados con el nombre nuevo.

Todos estos errores comparten algo en común: no son fallas de que no sepas escribir una función, son fallas de que el código, al moverse de lugar, perdió algo del contexto que tenía antes. Ese es, precisamente, el tipo de detalle que la verificación función por función está pensada para atrapar a tiempo.

✅ Cierra con un commit

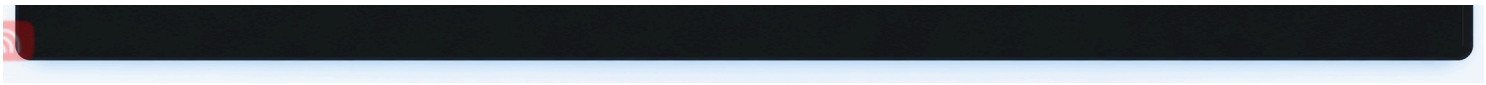
Con el resultado verificado, guarda este avance con un commit propio, distinto del que hiciste en el Bloque 5:

```
git add .
git commit -m "Refactorizar módulo en funciones separadas"
```

Ese mensaje deja un registro claro de qué cambió y por qué, algo que también forma parte de un buen historial de commits.

Un solo **commit** para todo este bloque es suficiente por ahora: no hace falta un commit por cada función que extraigas. Lo importante es que, cuando confirmes, el resultado ya esté verificado y el código viejo ya esté borrado, no a medio camino.





Siguiente lección

Bloque 6 · Refactorización del módulo

Con tu **módulo** ya dividido en funciones, verificado y confirmado en tu historial, la siguiente lección de este mismo bloque revisa el resultado completo: qué cambió exactamente, y por qué esta nueva versión queda mejor preparada para lo que viene en el **Bloque 7**.

Última modificación: jueves, 24 de septiembre de 2026, 15:21